Rule Placement and Switch Migration-based Scheme for Controller Load Balancing in SDN

Gengbiao Yue, Yumei Wang, and Yu Liu
School of Artificial Intelligence, Beijing University of Posts and Telecommunications, Beijing, China
Email: {yuegengbiao, ymwang}@bupt.edu.cn, liuyurainy@gmail.com

***Abstract*—In software-defined networks, due to the limited flow table capacity, unreasonable rule placement will cause the flow table overflow problem. These flows without flow rules installed need to be processed by controller, which increases and even unbalances controller load. Based on the average packet end-to-end delay, we propose a rule placement and switch migration-based scheme for controller load balancing. In the routing and rule placement phase, the Cost-Aware Routing (CAR) algorithm takes into account the flow table occupancy while utilizing the installed rules to alleviate flow table overflow and preliminarily balance the controller load. In the switch migration phase, the Benefit-Cost Switch Migration (BCSM) algorithm obtains the migration option with the maximum total benefit. Numerical results show that the CAR algorithm reduces and balances controller load to achieve lower delay than Random and FlowStat. And the BCSM algorithm balances the controller load and reduces packet delay than SMCS and ESMLB.**

***Index Terms*—SDN controller, routing, rule placement, switch migration, load balancing**

I. Introduction

Owing to its global network view and simplified management, the software-defined networks (SDN) is widely used in data center, wide area network and other fields. The placement of flow rules in SDN is mainly divided into wildcard, exact-match and hybrid [1]. There are visibility and security issues when using only wildcard rules. The Exact-match placement can improve visibility, but due to the capacity (about 1600 entries [2]) limitations of flow tables based on ternary content addressable memory (TCAM), it is difficult for the switch to install exact-match rules for all active flows (about 2000–5000 [3]) that pass through it. Therefore, the hybrid placement scheme, that is, the exact-match rule is only installed on one of the switches on the path, while installing the wildcard on the other switches, is a more practical rule placement scheme.

However, even if the hybrid placement scheme is used, the flow table is still easy to overflowed if the switch to install the exact-match rule is not reasonably selected, as shown in Fig. 1. Assuming that the flow table capacity of each switch is 2 entries, the installed wildcard rule W(1,2) can match both flow1 and flow2, and the exact-match rule E(1) can only match flow1. S3 cannot install exact-match rule for flow4 after installing rules for flow2 and flow3. For the flow with no rules installed, called overflowed flow, each of its packets is encapsulated as a Packet-In message and uploaded to controller for processing [4], which increases the controller load. This may overload C1 and cause unbalanced load between C1 and C2. The processing delay of the C1 will increase significantly, which in turn increase the packet end-to-end delay of the flow. In fact, when routing flow2, we can take advantage of the installed W(1,2) after routing flow1 to improve rule utilization and avoid flow table overflow. The switch migration protocol proposed in [5] can migrate the switch from the overloaded controller to the underloaded controller and reduce the load of overloaded controller. However, switch migration causes a lot of migration costs, some of which are considered in [6], [7]. Therefore, when selecting the switch to install the exact-match rule, the controller load unbalance caused by the overflowed flows should be considered.

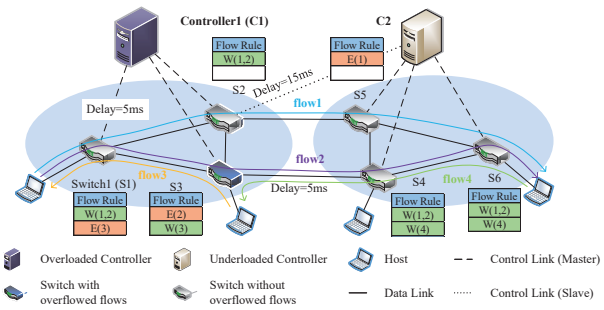

Fig. 1. SDN controller load unbalance scenario with limited flow table capacity.

In this paper, we establish a controller load balancing model which takes these overflowed flows into account, and formulate an average packet end-to-end delay optimization problem. And we propose a controller load balancing scheme based on flow rule placement and switch migration, including the cost-aware routing (CAR) algorithm and benefit-cost switch migration (BCSM) algorithm, to solve the optimization variables. Firstly, the switch cost is determined by considering whether there are installed flow rules that can be used directly, the occupancy of the flow table and the load of master controller. The forwarding path and the switch to install the exact-match rule are then selected by the CAR algorithm based on the switch cost. Finally, when migrating the switch to further balance the controller load, the benefits and various migration costs brought by switch migration are weighed in the BCSM algorithm to improve the migration efficiency.

The remainder of this paper is organized as follows. Section II discusses the existing works of rule placement and switch

2022 IEEE Symposium on Computers and Communications (ISCC) | 978-1-6654-9792-3/22/$31.00 ©2022 IEEE | DOI: 10.1109/ISCC55528.2022.9912885

978-1-6654-9792-3/22/$31.00 ©2022 IEEE

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:13:18 UTC from IEEE Xplore. Restrictions apply.

migration. Section III formulates the controller load balancing into an optimization problem. Section IV decomposes the optimization problem and introduces the proposed algorithms in details. Section V describes the experimental details and analyzes the effectiveness of the algorithms. Finally, Section VI concludes the paper.

II. Related Work

Rule placement is an important research problem in SDN when the capacity of flow table is limited. Bera et al. [1] proposed a heuristic algorithm which included three stages: path selection, rule placement and rule redistribution. The authors took the utilization rate of the flow table, the path capacity and the degree of the switch into account when selected the path and the switch to deploy exact-match rule. Luo et al. [8] proposed three algorithms for routing, rule caching and resource allocation to minimize end-to-end delay when multiple user flows compete for limited switch flow table capacity and link capacity. But they only considered the placement form of exact-match rules. Considering the limitation of the flow table capacity and processing capacity of the switch, Mondal et al. [9] used generalized Nash bargaining game to determine the rate assigned to the switch and used the heuristic algorithm to get the mapping between the flow rules and switches, which realized the load balance among the switches. But the authors only considered the single-hop scenario from the IoT device to the switch and ignored the routing. Zhao et al. [10] proposed a hybrid rule placement strategy called HiFi, which ensured that there was at least one switch with no wildcard rule on the flow path to achieve fine-grained flow management and minimized the number of flow tables required.

In order to cope with the unbalanced controller load, Dixit et al. [5] proposed a four-phase switch migration protocol and chose the switch migration option that minimized the variance of all controller loads. When migrating switch, three aspects of migration costs were considered in [6]. And the Packet-In messages from the migrated switches could not be immediately processed by controller during the migration, namely migration blackout problem, resulting in extra migration costs. Although the authors addressed this problem by temporarily routing newly arriving flow away from the migrated switches, this solution was not suitable for edge switches. Many researches have been made on reducing frequent switch migration and improving migration efficiency. In [11], the author used the response time of the controller to judge whether the controller was overloaded, which reduced the unnecessary migration caused by using the fixed threshold. In [12], an efficient switch migration-based load balancing (ESMLB) framework was introduced and used TOPSIS algorithm to jointly consider the load, RAM, hop count and bandwidth when selecting the target controller. However, it used a fixed threshold to determine whether the controller was overloaded.

III. Problem Formulation

We consider the whole software-defined networks as a graph $\mathcal{G} = (\mathcal{S}, \mathcal{E})$, where $\mathcal{S} = \{S_1, \ldots, S_M\}$ is the set of switches and $\mathcal{E} = \{(i,j) | S_i, S_j \in \mathcal{S}\}$ is the set of links. Suppose that the flow table capacity of the switch S_m is Ψ_m. The set of controllers in SDN is $\mathcal{C} = \{C_1, \ldots, C_N\}$, and the processing power of controller C_n is Φ_n in terms of the number of packets it can handle per second. Once the load of the controller exceeds this value, its processing delay will increase sharply. The mapping between the controller C_n and the switch S_m is denoted by the binary variable $x_{mn} = \{0, 1\}$, which is 1 if the master controller of switch S_m is C_n. $\mathcal{F} = \{f_1, \ldots, f_F\}$ denotes the set of flows in the network. We use $\langle S_s^f, S_d^f, Q^f, R_f \rangle$ to denote a unique flow $f \in \mathcal{F}$, and they represent the source IP address, destination IP address, port number and packet rate of f, respectively. Table I summarizes the used notations.

TABLE I: List of Notations

Notation	Description
C_n	n^{th} controller
S_m	m^{th} switch
Φ_n	Processing power of nth controller
Ψ_m	Flow table capacity of mth switch
f	A flow in the network
R_f	Packet rate of f
$L_{S_m}^{new}$	Arrival rate of new flows on S_m
L_{S_m}	Load of switch S_m on its master controller
L_{C_n}	Load of controller C_n
$\hat{T}_n$	Processing delay of controller C_n
$\tilde{T}_{ij}$	Propagation delay of link (i,j)
$\tilde{T}_{mn}$	Propagation delay between switch S_m and controller C_n

The path of f is denoted by $P_f = \{p_{ij}^f | (i,j) \in \mathcal{E}\}$. Hence, whether the flow f passes through the switch S_m can be represented as:

$$y_m^f = \begin{cases} 0, \sum_{(i,m)\in\mathcal{E}} p_{im}^f + \sum_{(m,j)\in\mathcal{E}} p_{mj}^f = 0 \\ 1, \sum_{(i,m)\in\mathcal{E}} p_{im}^f + \sum_{(m,j)\in\mathcal{E}} p_{mj}^f \geq 1 \end{cases} \tag{1}$$

In addition, $z_m^f = e_m^f + w_m^f = \{0, 1\}$ is used to indicate whether the rule of f can be installed in S_m. Where e_m^f and w_m^f indicate whether to install the exact-match rule or the wildcard rule, respectively.

The load generated by each switch on its master controller mainly includes: (1) When a new flow arrives, the source switch needs to send a Packet-In message to the controller to process the flow. So this type of load is equal to the arrival rate of the new flow $L_{S_m}^{new}$; (2) When the switch flow table is full, packets of flows that pass through the switch but have no flow rules installed also need to be processed by the controller. Then we can get the load generated by S_m to its master controller as:

$$L_{S_m} = L_{S_m}^{new} + \sum_{f\in\mathcal{F}} R_f \cdot y_m^f \cdot \left(1 - z_m^f\right) \tag{2}$$

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:13:18 UTC from IEEE Xplore. Restrictions apply.

Therefore, the total load of the controller C_n can be calculated as:

$$L_{C_n} = \sum_{m=1}^{M} L_{S_m} \cdot x_{mn} \tag{3}$$

According to the measurement in [11], the processing delay $\hat{T}_n$ of C_n can be obtained according to its load.

So the packet end-to-end delay of flow f is:

$$T_f = \sum_{(i,j)\in\mathcal{E}} \tilde{T}_{ij} \cdot p_{ij}^f + \sum_{m=1}^{M}\sum_{n=1}^{N} x_{mn} \cdot y_m^f \cdot \left(1 - z_m^f\right) \cdot T_{mn} \tag{4}$$

The first term of Equation (4) is the propagation delay among switches in the flow path. And the second item is the total delay $T_{mn} = 2\tilde{T}_{mn} + \hat{T}_n$ from switch S_m to controller C_n when the flow rules are not installed, including switch-to-controller propagation delay $\tilde{T}_{mn}$ and controller processing delay $\hat{T}_n$.

Different from the optimization of load balancing rate in [12], in order to make the result of controller load balancing beneficial to the delay, we take the average packet delay as optimization objective.

$$\bar{T} = \frac{\sum_{f\in\mathcal{F}} T_f \cdot R_f}{\sum_{f\in\mathcal{F}} R_f} \tag{5}$$

As a result, the optimization problem is obtained:

$$\min_{p,e,w,x} \bar{T} \tag{6}$$

$$\text{s.t.} \sum_{f\in\mathcal{F}} z_m^f \le \Psi_m, \forall S_m \in \mathcal{S} \tag{6a}$$

$$\sum_{n=1}^{N} x_{mn} = 1, \forall S_m \in \mathcal{S} \tag{6b}$$

$$\sum_{(i,m)\in\mathcal{E}} p_{im}^f - \sum_{(m,j)\in\mathcal{E}} p_{mj}^f = \begin{cases} -1, & \text{if } S_m = S_s^f \\ 1, & \text{if } S_m = S_d^f \\ 0, & \text{if } S_m \ne S_s^f \text{ and } S_m \ne S_d^f \end{cases}, \forall S_m \in \mathcal{S}, \forall f \in \mathcal{F} \tag{6c}$$

$$z_m^f \le y_m^f, \forall S_m \in \mathcal{S}, \forall f \in \mathcal{F} \tag{6d}$$

$$\sum_{m=1}^{M} \left(e_m^f + \left(1 - z_m^f\right)\right) \ge 1, \forall f \in \mathcal{F} \tag{6e}$$

Constraint (6a) ensures that the number of rules installed in the switch S_m does not exceed its flow table capacity Ψ_m. Constraint (6b) indicates that the switch is associated with only one master controller. Constraint (6c) represents the limitation of routing variables. For all switches except the source switch S_s^f and the destination switch S_d^f, the number of incoming and outgoing edges belonging to the path is equal. Constraint (6d) indicates that if the flow rule of f can be installed in switch S_m, the prerequisite is that f passes through that switch. Constraint (6e) indicates that for each flow, there is at least one switch with exact-match rule or one switch without flow rule installed. The flow with no flow rule installed must pass through the controller, which can achieve functions similar to exact-match rule in the sense of statistics and security.

IV. Problem Analysis And Solutions

Although above problem can be solved by optimizer (e.g. GUROBI), its high time complexity is not suitable for scenarios where traffic fluctuates rapidly. Therefore, we decompose the problem and use a rule placement and switch migration-based scheme for controller load balancing to get the optimization variables. For each flow f, the Cost-Aware Routing (CAR) algorithm is used to to solve the routing variables p_{ij}^f and select a switch to install the exact-match rule on the path. After initializing the switch-controller mapping variables x_{mn} using the existing controller placement scheme [13], we periodically run the Benefit-Cost Switch Migration (BCSM) algorithm to re-adjust these mapping variables.

A. Routing and rule placement

Algorithm 1 is used to select the forwarding path and install the rules. Firstly, the Yen's algorithm [14] is used to obtain K shortest paths in advance according to the path propagation delay. Then, the path with lowest path cost is selected as the forwarding path. The path cost $PathCost_f^i$ includes the path propagation delay cost (Step-4) and the switch cost (Step-7).

The switch cost is obtained through the function GET_SW_COST. In this function, the switch cost is -1 when it has a wildcard rule that can match f. In this way, it tends to use the installed rules to forward f and reduce the number of wildcard rules required. For switches where there is no available wildcard rule, the more the flow table is occupied, the more likely it is to overflow. Therefore, the switch with less flow table occupation should be selected. In addition, when the switch flow table is full, the flow needs to be processed by the controller, resulting in switch-to-controller delay T_{mn}. Therefore, the switch with less T_{mn} should be selected. Owing to T_{mn} includes the controller processing delay $\hat{T}_n$ that reflects the controller load, a smaller T_{mn} will correspond to a smaller controller load. Choosing a controller with a lower load can initially balance the controller load.

Owing to there is wildcard rule that can be used directly on that switch, when choosing the switch to install exact-match rule, we ignore the switch with a switch cost of -1, as shown in Step-9. After the best path P_f^b is determined, the corresponding exact-match switch $ExactSw_f^b$ is also determined. Hence, we use the function INSTALL_RULES to install the rules. Based on the fact that the wildcard rules can match more flows than the exact-match rules, when the capacity of the flow table is insufficient, we install the wildcard rule preferentially and delete the exact-match rule to improve the utilization of the flow table, as shown in Step 33-35.

B. Dynamic switch migration

In order to reduce the average packet delay through migrating switch, we determine whether the switch should be

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:13:18 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Cost-Aware Routing Algorithm

Input: Network Graph $\mathcal{G}$, Set of flows $\mathcal{F}$
Output: Set of paths $\{P_f, \forall f \in \mathcal{F}\}$

Initialize: $\{\mathcal{P}_f^{can}\}$ ←shortest K candidate paths of $f \in \mathcal{F}$
for $f \in \mathcal{F}$ **do**
  for $P_f^i \in \mathcal{P}_f^{can}$ **do**
    $PathCost_i \leftarrow$ path propagation delay
    for $S_m \in P_f^i$ **do**
      $SwCost_m = \text{GET_SW_COST}\,(S_m, f)$
      $PathCost_i = PathCost_i + SwCost_m$
    end for
    $ExactSw_f^i \leftarrow$ get switch with the min $SwCost$ if $SwCost_m \geq 0$ in P_f^i
  end for
  $P_f^b \leftarrow$ get path with the min $PathCost$ in $\mathcal{P}_f^{can}$
  $\text{INSTALL_RULES}\left(P_f^b, ExactSw_f^b, f\right)$
end for
function $\text{GET_SW_COST}(S_m, f)$
  if S_m has wildcard rule of f **then**
    return -1
  else
    $Cost_m = (\text{number of rules in } S_m)\,/\Psi_m$
    if S_m has no empty entry **then**
      $Cost_m \leftarrow Cost_m + T_{mn}$
    end if
    return $Cost_m$
  end if
function $\text{INSTALL_RULES}(P_f^b, ExactSw_f^b, f)$
  for $S_m \in P_f^b$ **do**
    if S_m is $ExactSw_f^b$ and has empty entry **then**
      Install exact-match rule of f
    end if
    if S_m is not $ExactSw_f^b$ **then**
      if S_m has empty entry **then**
        Install wildcard rule of f
      else if S_m has exact-match rule **then**
        Delete one of the exact-match rule in S_m
        Install wildcard rule of f
      end if
    end if
  end for

migrated and how to migrate based on the trade-off between benefits and costs.

Assuming that the benefit of migrating the switch S_m from C_n to C_d is B_m.

$$B_m = \left(\hat{T}_n - \hat{T}_n^*\right) \cdot L_{C_n}^* \tag{7}$$

where $\hat{T}_n^*$ is the processing delay of C_n after the migration is completed, which is determined by the controller load $L_{C_n}^* = (L_{C_n} - L_{S_m})$. This benefit comes from the load reduction of the source controller C_n, which reduces the processing delay of all packets processed by the controller.

We consider four aspects of migration costs.

(1) The change of the processing and propagation delay for the flows without flow rules installed in the switch S_m :

$$Cost_1 = \left(\hat{T}_d^* - \hat{T}_n\right) \cdot L_{S_m} \tag{8}$$

$$Cost_2 = 2 \cdot \left(\tilde{T}_{md} - \tilde{T}_{mn}\right) \cdot L_{S_m} \tag{9}$$

where $\hat{T}_d^*$ is the processing delay of the target controller C_d after migration, which is determined by its load $L_{C_d}^* = L_{C_d} + L_{S_m}$.

(2) The change of the processing delay for the flows without flow rules installed in other switches controlled by C_d:

$$Cost_3 = \left(\hat{T}_d^* - \hat{T}_d\right) \cdot L_{C_d} \tag{10}$$

(3) The migration blackout problem makes the flow in the switch S_m unable to process by the controller, and the migration time is generally 30ms [5]. The migration blackout cost is defined as:

$$Cost_4 = 30 \cdot L_{S_m} \tag{11}$$

We can get the total benefit of migrating S_m from C_n to C_d:

$$TB_{md} = B_m - Cost_1 - Cost_2 - Cost_3 - Cost_4 \tag{12}$$

Algorithm 2 Benefit-Cost Switch Migration Algorithm

Input: $\{L_{S_m}, \forall S_m \in \mathcal{S}\}, \{L_{C_n}, \forall C_n \in \mathcal{C}\}$
Output: Switch migration action

Initialize: $Max_total_benefit \leftarrow 0$
for $S_m \in \mathcal{S}$ **do**
  for $C_d \in \mathcal{C}$ **do**
    Calculate TB_{md} use Eq. (12)
    if $TB_{md} > Max_total_benefit$ **then**
      $Max_total_benefit \leftarrow TB_{md}$
      $S_m^{'} \leftarrow S_m, C_d^{'} \leftarrow C_d$
    end if
  end for
end for
if $Max_total_benefit > 0$ **then**
  Migrate $S_m^{'}$ to $C_d^{'}$
end if

The Algorithm 2 runs periodically to obtain the migration choice of max total benefit. When the maximum total benefit is greater than 0, the switch migration protocol proposed in [5] is used to migrate switch.

V. Performance Evaluation

In this section, we evaluate the performance of CAR and BCSM. Firstly, we describe the simulation setup, including parameter settings, comparison schemes, and performance metrics. Then, we show the numerical results and analysis.

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:13:18 UTC from IEEE Xplore. Restrictions apply.

A. Simulation setup

We use python libraries such as NetworkX [15] and NumPy [16] to build the network topology and generate traffic. We verify the performance of the algorithm in the k-ary Fat-tree topology which commonly used in data center networks and set k to $\{4, 6, 8, 10, 12\}$. Each edge switch connects to $k/2$ servers with different IP addresses. The number of active flows per edge switch is 1000–5000, and the packet rate of each flow is 2–5 packets/s [3]. The flow table capacity of each switch is set to 1600 entries [2] and the link propagation delay between switches is 5ms. The switch-to-controller propagation delay is set to $\{5, 15, 25, ...\}$ms depends on the deployment distance, respectively, as labelled in Fig. 1. Each controller controls 5 switches, so there are $(k/2)^2$ controllers in the k-ary Fat-tree topology. The processing power of each controller is set to 18000 requests per second [17].

For the routing scheme, we use two comparison schemes: (1) Random, which randomly selects one of the K shortest paths and randomly selects a switch to install the exact-match rule; (2) FlowStat [1], which considers the utilization of the flow table and the degree of the switch when selecting the path and the exact switch. Since we ignore the limitation of link capacity, we set the parameters of FlowStat to $\alpha = 1, \beta = 0$.

For the switch-to-controller mapping scheme, we also use two comparison schemes: (1) Static mapping between controller and switch (SMCS), that is, no switch migration; (2) ESMLB [12], once the controller load exceeds the set threshold, the switch with the least load under the overloaded controller is migrated. When selecting the target controller, the CPU and RAM usage of the controller and the number of hops from switch to controller are taken into account.

We verify the effectiveness of the proposed scheme in terms of following performance indicators: (1) Maximum and average controller load; (2) Average packet end-to-end delay, as shown in Section III; (3) Load balancing rate (LBR), a higher value means a more balanced load between controllers;

$$LBR = 1 - \frac{\sum_{n=1}^{N} |L(C_n) - \overline{L}|}{N \cdot \overline{L}} \tag{13}$$

(4) Number of switch migrations. We consider the switch migration cost caused by migration blackout problem, so the fewer migrations, the lower the migration costs.

B. Numerical Results

As shown in Figure 2, the maximum and average controller loads increase as k increases in the three routing schemes. As k increases, the number of active flows in the network increases, and the limited flow table capacity is more unable to meet the required number of rules. The overflowed flows increase controller load. FlowStat chooses the switch with the largest remaining flow table capacity and reduces the possibility of flow table overflow compared with Random, thus reducing the maximum controller load. However, because the installed rules are not fully utilized, its average controller load is similar to Random. By making use of the installed wildcard rules, CAR significantly reduces the required number of flow rules and reduces the possibility of flow table overflow, thus reducing average and maximum controller load.

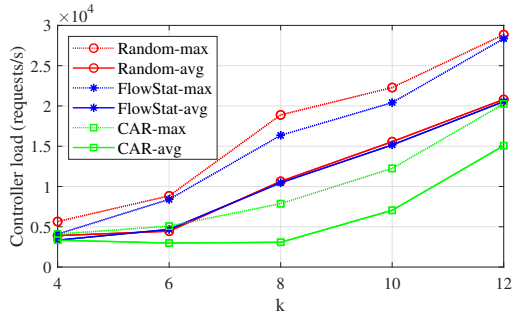

Fig. 2. Maximum and average controller load with different k-ary topology.

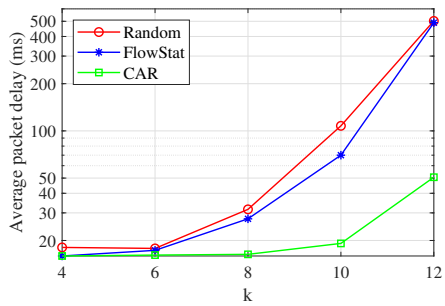

Fig. 3. Average packet end-to-end delay with different k-ary topology.

Figure 3 shows the average packet end-to-end delay. Since CAR has a lower controller load, the controller processing delay is reduced, thereby reducing the packet end-to-end delay.

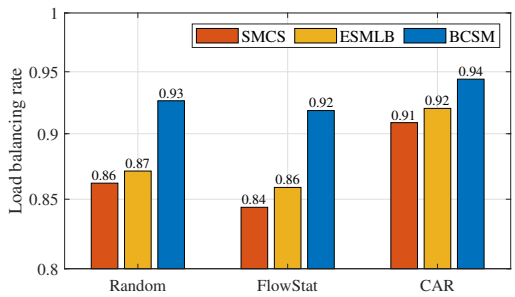

Fig. 4. Load balancing rate when $k = 12$.

Figure 4 is the LBR for different routing schemes and switch-to-controller mapping schemes when $k = 12$. For routing schemes, Random and FlowStat do not consider the load of the controller when the flow table overflows, resulting in an unbalanced controller load. CAR preliminarily balances controller loads by alleviating the flow table overflow and selecting the switch with less load on its master controller when the overflow occurs. For migration schemes, by weighing the benefits and costs of migration, BCSM achieves a higher

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:13:18 UTC from IEEE Xplore. Restrictions apply.

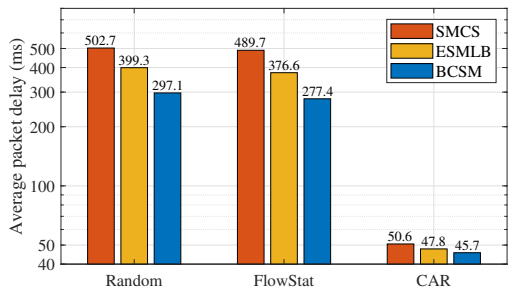

Fig. 5. Average packet end-to-end delay when $k = 12$.

LBR than SMCS and ESMLB that uses a fixed threshold to determine migration.

Figure 5 is the average packet end-to-end delay of different routing schemes and switch-to-controller mapping schemes when $k = 12$. For routing schemes, Random and FlowStat have larger packet delay than CAR due to larger and more unbalanced controller load. For migration schemes, BCSM can achieve a more balanced controller load, thus reducing the controller processing delay.

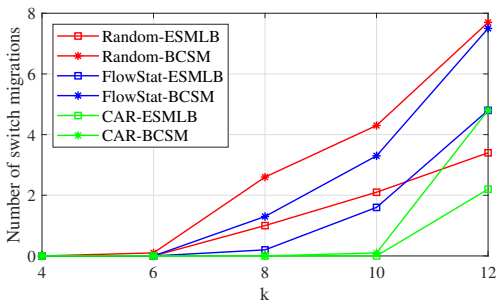

Fig. 6. Number of switch migrations with different k-ary topology.

Figure 6 shows the number of switch migrations under different topologies. It can be found that CAR significantly reduces the number of switch migrations by reducing the load on the controller. In addition, although BCSM has more switch migrations than ESMLB, the increase is acceptable because of its lower delay.

To sum up, by using the CAR algorithm and BCSM algorithm together, we can achieve lower average packet end-to-end delay with a small number of switch migrations.

VI. Conclusion

In this paper, we propose a controller load balancing scheme based on rule placement and switch migration to reduce and balance the controller load. To solve the problem of flow table overflow due to the limited capacity of the switch flow table, the CAR algorithm selects the forwarding path and the switch to install the exact-match rule by taking advantage of the installed flow rules and considering the flow table occupancy and controller load. Further, the BCSM algorithm chooses efficient migration options by weighing benefits against costs. Numerical simulation on the Fat-tree topology shows that, compared with existing rule placement schemes – Random and FlowStat, and switch-to-controller mapping schemes – SMCS and ESMLB, our CAR algorithm and BCSM algorithm can reduce and balance controller load and achieve lower average packet end-to-end delay.

Acknowledgment

This work was supported by National Key R&D Program of China (No.2018YFB1800501). The authors would like to thank the reviewers for their constructive comments.

References

[1] S. Bera, S. Misra and A. Jamalipour, "FlowStat: Adaptive Flow-Rule Placement for Per-Flow Statistics in SDN," IEEE Journal on Selected Areas in Communications, vol. 37, no. 3, pp. 530-539, Mar. 2019.
[2] T. Cheng, K. Wang, L. Wang and C. Lee, "An In-Switch Rule Caching and Replacement Algorithm in Software Defined Networks," 2018 IEEE International Conference on Communications (ICC), 2018, pp. 1-6.
[3] T. Benson, A. Akella, and D. A. Maltz, "Network traffic characteristics of data centers in the wild," in Proceedings of the 10th ACM SIGCOMM conference on Internet measurement, 2010, pp, 267–280.
[4] OpenFlow switch specification 1.5.1 [online], Available: https://opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v1.5.1.pdf
[5] A. Dixit, F. Hao, S. Mukherjee, T. V. Lakshman and R. R. Kompella, "ElastiCon: an elastic distributed SDN controller," 2014 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS), 2014, pp. 17-27.
[6] Y. Liu, H. Gu, F. Yan and N. Calabretta, "Highly-Efficient Switch Migration for Controller Load Balancing in Elastic Optical Inter-Datacenter Networks," IEEE Journal on Selected Areas in Communications, vol. 39, no. 9, pp. 2748-2761, Sept. 2021.
[7] Y. Xu et al., "Dynamic Switch Migration in Distributed Software-Defined Networks to Achieve Controller Load Balance," IEEE Journal on Selected Areas in Communications, vol. 37, no. 3, pp. 515–529, Mar. 2019.
[8] L. Luo, R. Chai, Q. Yuan, J. Li and C. Mei, "End-to-End Delay Minimization-Based Joint Rule Caching and Flow Forwarding Algorithm for SDN," IEEE Access, vol. 8, pp. 145227-145241, 2020.
[9] A. Mondal and S. Misra, "FlowMan: QoS-Aware Dynamic Data Flow Management in Software-Defined Networks," IEEE Journal on Selected Areas in Communications, vol. 38, no. 7, pp. 1366-1373, Jul. 2020.
[10] G. Zhao, H. Xu, J. Fan, L. Huang, and C. Qiao, "Achieving Fine-Grained Flow Management Through Hybrid Rule Placement in SDNs," IEEE Transactions on Parallel and Distributed Systems, vol. 32, no. 3, pp. 728–742, Mar. 2021.
[11] J. Cui, Q. Lu, H. Zhong, M. Tian and L. Liu, "A Load-Balancing Mechanism for Distributed SDN Control Plane Using Response Time," IEEE Transactions on Network and Service Management, vol. 15, no. 4, pp. 1197-1206, Dec. 2018.
[12] K. S. Sahoo et al., "ESMLB: Efficient Switch Migration-Based Load Balancing for Multicontroller SDN in IoT," in IEEE Internet of Things Journal, vol. 7, no. 7, pp. 5852-5860, July 2020.
[13] B. Heller, R. Sherwood, and N. McKeown, "The controller placement problem," ACM SIGCOMM Comput. Commun. Rev., vol. 42, no. 4, pp. 473–478, Sep. 2012.
[14] J. Y. Yen, "Finding the K shortest loopless paths in a network," Management Science, vol. 17, no. 11, pp. 712–716, Jul. 1971.
[15] Network Analysis in Python. [Online]. Available: https://networkx.org/
[16] Scientific Computing with Python. [Online]. Available: https://numpy.org/
[17] P. Berde et al., "ONOS: Towards an open, distributed SDN OS," Proc. 3rd Workshop Hot Topics Softw. Defined Netw., 2014, pp. 1–6.

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:13:18 UTC from IEEE Xplore. Restrictions apply.